



Intro to Deep Reinforcement Learning

Blink Sakulkueakulsuk

Introduction to Deep Reinforcement Learning

What is deep reinforcement learning?

Core concepts

Agent components

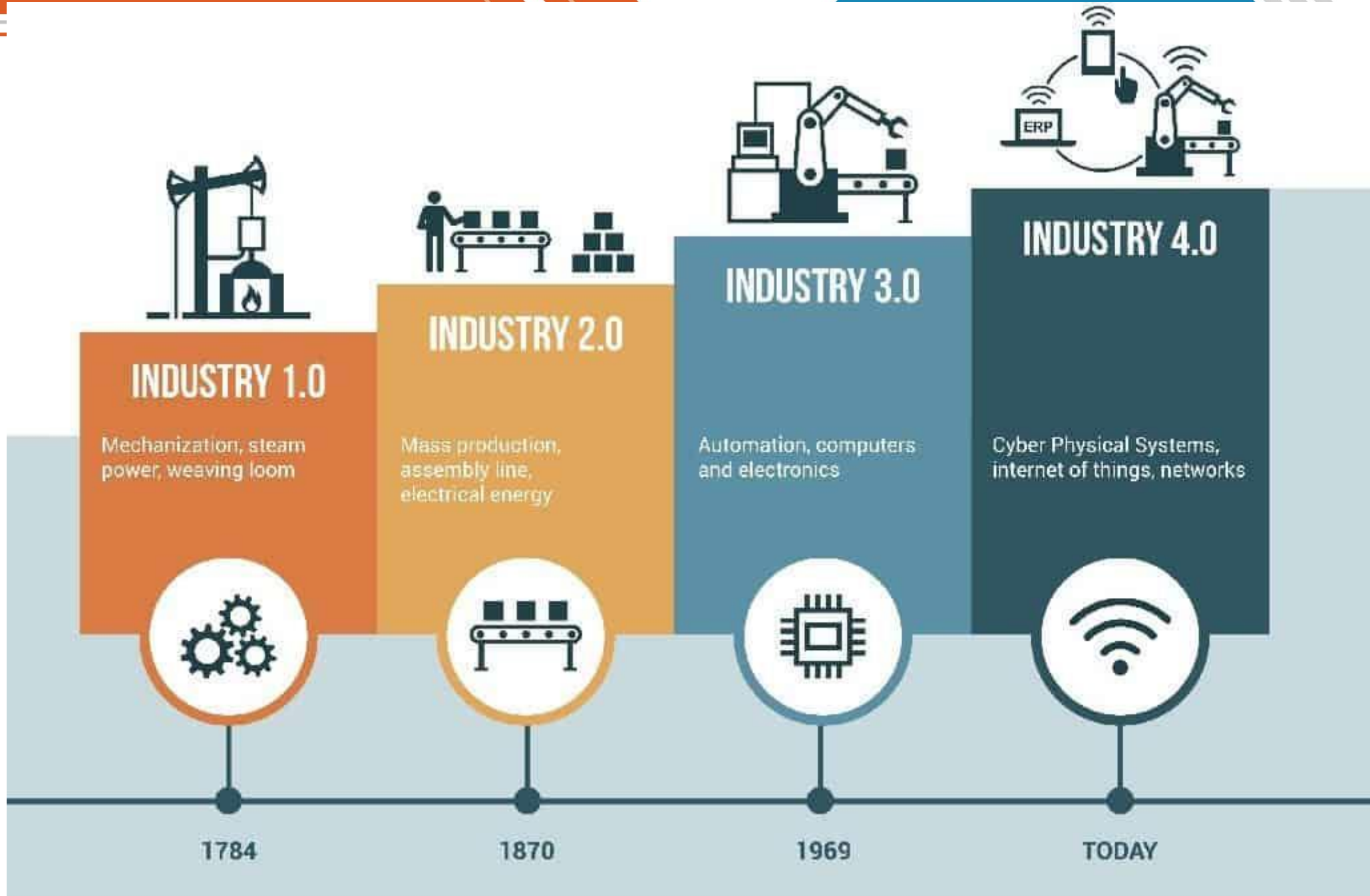
Challenges in reinforcement learning

Introduction to Deep Reinforcement Learning

Deep reinforcement learning combines **deep learning** to solve some problems that **traditional reinforcement learning** techniques cannot solve.

We need to learn what reinforcement learning is first.

What is Reinforcement Learning?



The next step is for the machine to make decisions
by themselves. . . But how?

2012 ALEXNET



PERCEPTION AI

SPEECH RECOGNITION
DEEP RECSYS
MEDICAL IMAGING

GENERATIVE AI

DIGITAL MARKETING
CONTENT CREATION

AGENTIC AI

CODING ASSISTANT
CUSTOMER SERVICE
PATIENT CARE

PHYSICAL AI

SELF-DRIVING CARS
GENERAL ROBOTICS

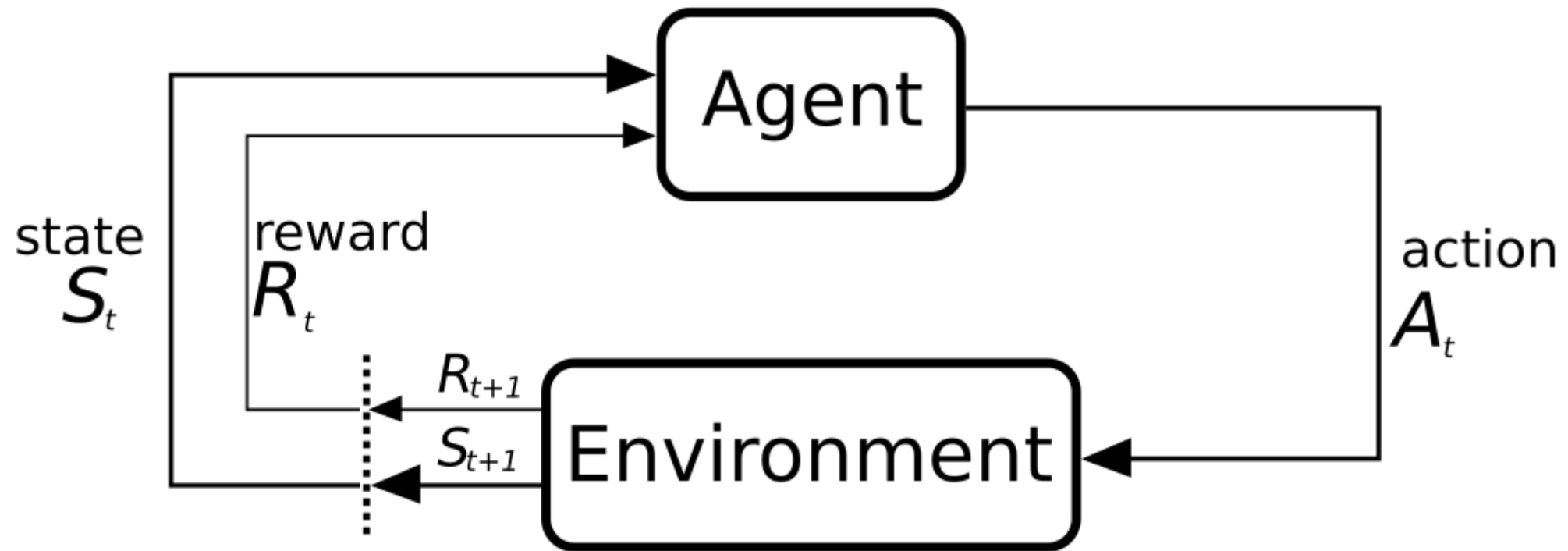


Reinforcement Learning

A way to learn – a natural one

- Learning through interactions
- Active learning
- Sequential interaction
- Goal-directed
- Learn even without the optimal sequence of actions

Reinforcement Learning

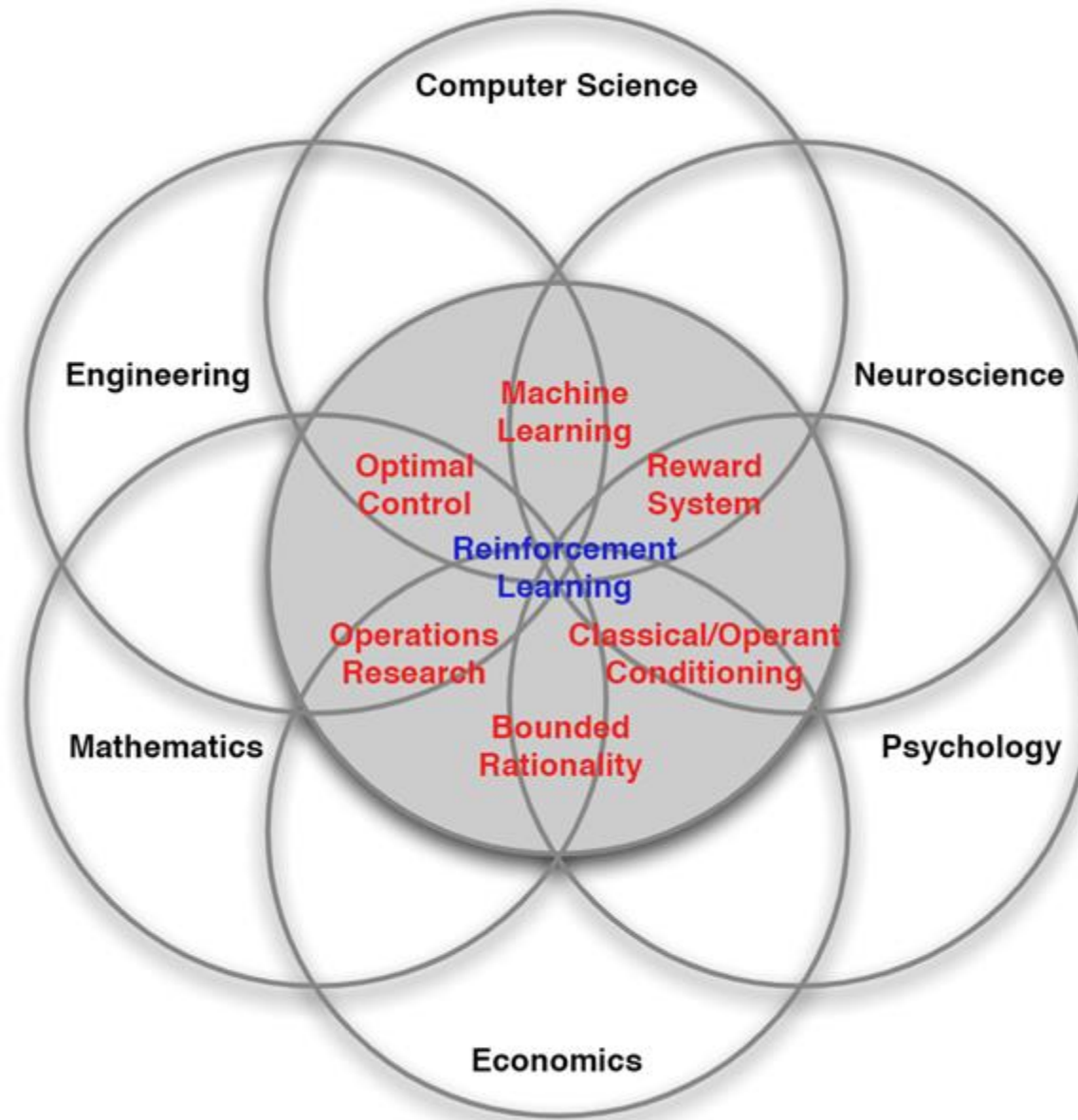


Why RL?

- To learn is to find solutions
- RL can find a general strategy (optimal or not) to solve a problem.
- RL can also learn online (learning while acting).

So . . . What is RL?

- A field of creating an **agent** that can learn to **make decisions** from its **interactions** with the **environment**. In many cases, the agent can learn **without examples**.
- In order to do this, these factors must be considered
 - Agent, environment, and action
 - Goal, time, and rewards
 - Experience, and optimal action



RL? What's the differences?

RL? What's the differences?

- **Active** Learning, rather than passive learning
- Often encountering **sequential interactions**
- **Goal-directed**. No class or label, just a reward for an action.
- Can learn from **zero to hero**
- Feedback doesn't have to be immediate.
- **Online** learning.

Let's Imagine

In this step, we will get you familiar with DRL at the very high abstraction level.

Let's do a thought experiment.

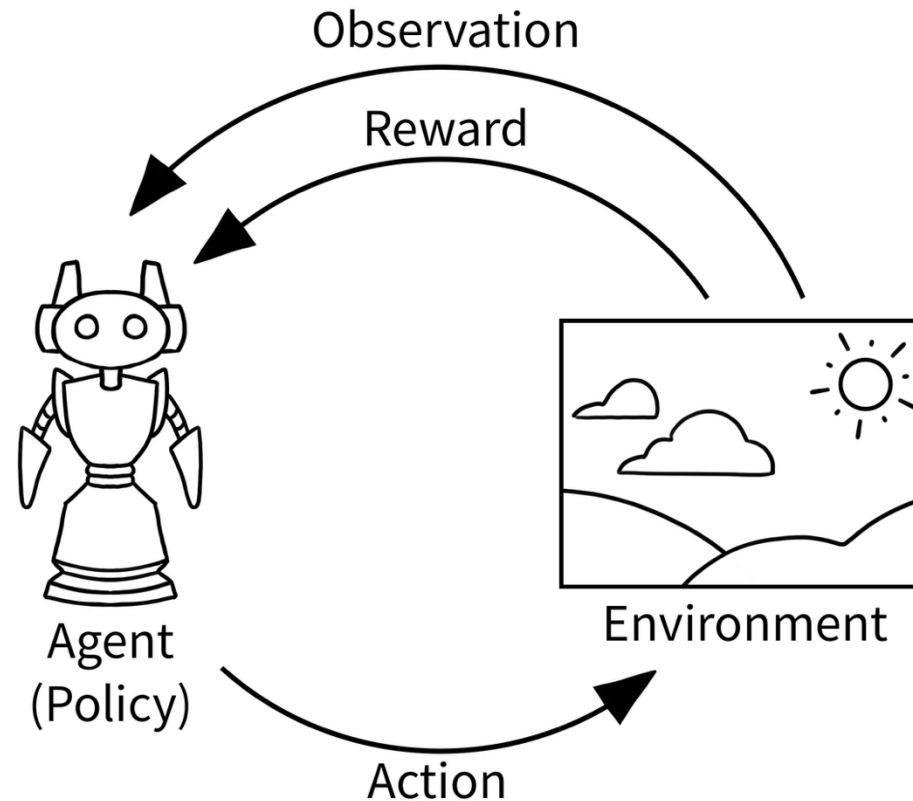
Formalizing Reinforcement Learning Problem

General RL Cycle

At each timestep t ,

The agent

- Get observation O_t
- Get reward signal R_t
- Take action A_t



The environment

- Get action A_t
- Give observation O_{t+1}
- Give reward signal R_{t+1}

Rewards

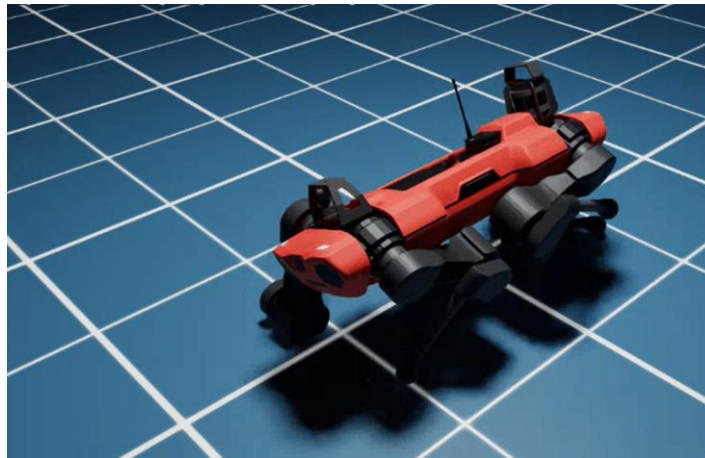
- A **reward** R_t is a scalar feedback signal
- It means, at time t , how good is the agent's action
- The ultimate goal of the agent is to maximize cumulative reward

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

- This is called **return**
- For RL, the agent tries to **learn whatever actions** to take to get as high reward as possible (reward maximization).

Reward Hypothesis

All of what we mean by **goals and purposes** can be well thought of as **maximization** of the **expected value** of the **cumulative sum** of a received scalar **signal (reward)**



Reward Example

Chess Agent	=>	Win, Score, ...
Self-Driving Car	=>	Distance, Safety, Efficiency, ...
Bipedal Robot Walking	=>	Distance, Speed, Stability, ...
Stock Investment	=>	Gains, Profits, ...

Values

- The agent can be in different states, and each states has different cumulative reward.
- We define the expected cumulative reward of a state s (expected return), the **value**.

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s] \end{aligned}$$

Wait, What the heck is that E thingy?

Expected Value

Expected value is a generalization of the weighted average.

Given a random variable X with a list of outcome $x_1, \dots, x_k$ with probability $p_1, \dots, p_k$ respectively, the expectation of X is

$$\mathbb{E}(X) = x_1p_1 + x_2p_2 + \dots + x_kp_k$$

For example, an expected value of a die roll is

$$\mathbb{E}(X) = 1\left(\frac{1}{6}\right) + 2\left(\frac{1}{6}\right) + 3\left(\frac{1}{6}\right) + 4\left(\frac{1}{6}\right) + 5\left(\frac{1}{6}\right) + 6\left(\frac{1}{6}\right) = 3.5$$

Values

- The agent can be in different states, and each states has different cumulative reward.
- We define the expected cumulative reward of a state s (expected return), the **value**.

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s] \end{aligned}$$

- This implies that, RL agent wants to choose an action that can maximize the value.

Values

- Since

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

- We can rewrite the equation as

$$G_t = R_{t+1} + G_{t+1}$$

- So the value becomes

$$v(s) = \mathbb{E}[R_{t+1} + v(S_{t+1}) \mid S_t = s]$$

Action, action, action, ...

- Since the agent wants to maximize the value, the agent needs some strategies to choose actions.
 - Long-term vs short-term reward
 - Bad start may lead to terrible outcomes, or it can give more long-term reward
- An agent needs to learn some way to tell the action from a state (a mapping from state to action). This is called a **policy**.

Action values

- Since an agent can have many actions, mapping values from state alone may not be sufficed
- We can map value to both action and state

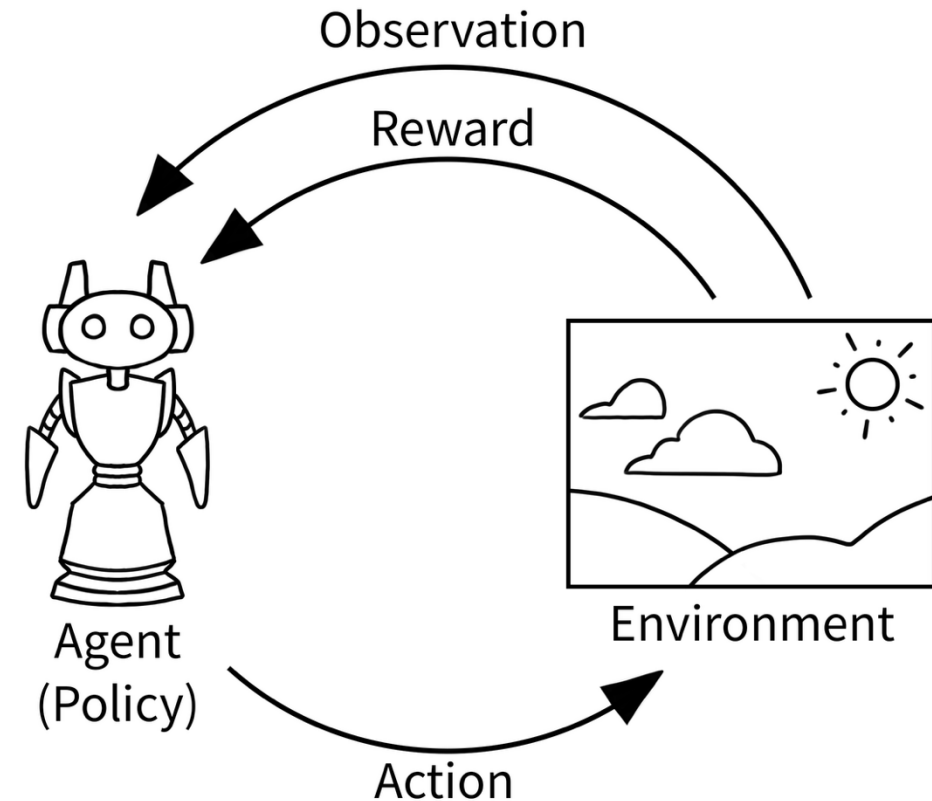
$$\begin{aligned} q(s, a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s, A_t = a] \end{aligned}$$

- We will discuss this (a lot) later

General RL Cycle

To formalize reinforcement learning problem, we need to understand

- Environment (dynamic of the problem)
- Reward signal (the goal)
- Agent (our actor)



Agent

- An agent consists of
 - **Agent States**
 - Policy
 - Value function (maybe)
 - Model (maybe)

State

Don't get mixed up between **agent state** and **environment state**

- Agent state represents a state of the agent. It can include information that the agent perceives.
- Environment state represents an internal state of the environment. Usually this is invisible to the agent and may contain a lot of irrelevant information for the agent.

For example, an environment state may be the full terrain of a simulation, an agent state is what a robot can see.

Both may or may not be the same.



History

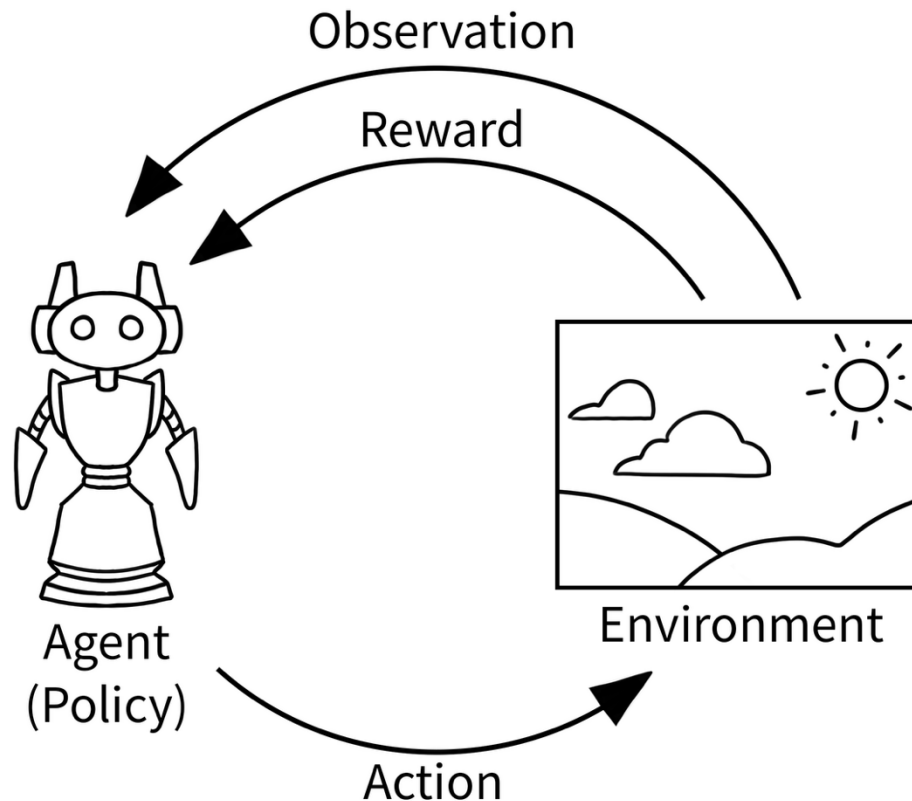
A sequence of observations, actions, and reward

$$H_t = O_0, A_0, R_1, O_1, \dots, O_{t-1}, A_{t-1}, R_t, O_t$$

One way to think of this is a log file of a robot simulation.

If we know the full history, we can construct an agent state S_t

Environment



The environment itself can also be complex, in term of observability

- Fully Observable Environment
- Partially Observable Environment

We will need to set up the problem differently so the agent can learn

Fully Observable Environment

- A very special (and not so common) case!
- The agent can see everything in the environment
 - Observation = environment state
 - $S_t = O_t =$ environment state
- If this is the case, the agent is in a **Markov decision process**

Markov decision processes

$$p(r, s \mid S_t, A_t) = p(r, s \mid H_t, A_t)$$

The equation above essentially means **the current state contains all information an agent needs to know regardless of the history**

MDP provides baseline for reinforcement learning.

- If the problem is Markov, we can use an agent state to represent the history, and throw the history away
- Other observations are Markov, but we don't use.
 - Full environment + agent state is too large
 - Full history keeps growing
- S_t is the agent state, not the environment state

Partially Observable Environments

An agent can perceive the environment partially

- Definitely not Markov
- The agent receives partial information
 - A robot sees the environment around itself but not the location
 - Card games that an agent doesn't see other players' cards

The environment can still be Markov, but the agent doesn't know it

We may be able to work around this to construct a Markov agent state. More on this later.

Agent State

A state of the agent in this case is **a function of history**

$$S_{t+1} = u(S_t, A_t, R_{t+1}, O_{t+1})$$

where u is a state update function

The agent state is typically much smaller than the environment state

Agent State

The full environment state of a maze



Agent State

A potential observation



An observation in a different location



Agent State

A potential observation



An observation in a different location



Are these two states Markov? Why or why not?

Partially observable environment

- We can modify the agent state
 - Adding last state to the state
 - Adding full history to the state
- What are the problems?

Partially observable environment

We can modify the agent state

- Adding last state to the state
- Adding full history to the state

What are the problems?

Building an MDP model may not be feasible.

- Sometimes, we cannot find a perfect MDP to solve
- However, we need to pick a good observation framework for an agent to learn a good policy or value prediction, ones that may not be optimal but good enough.

Agent

- Agent consists of
 - States
 - **Policy**
 - Value function
 - Model

Policy

- Given a state, a policy $\pi(S)$ defines an agent's action
 - Mapping from agent state to action
- Deterministic policy: $\pi(S) = A$
- Stochastic policy: $\pi(A|S) = p(A|S)$

Wait again, what the heck is that vertical line?

Conditional Probability

Conditional Probability tells us a likelihood of an outcome under some specific circumstances. For example,

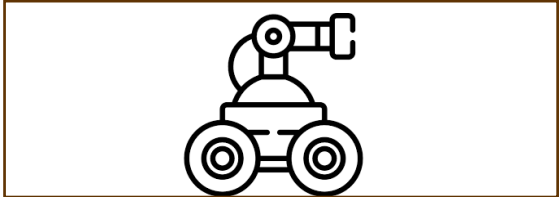
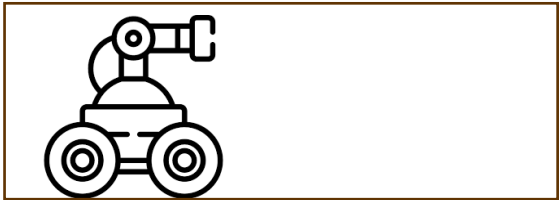
$$P(Yes) = \frac{150}{200} = 0.75$$

$$P(Yes|A) = \frac{50}{80} = 0.625$$

$$P(A|Yes) = \frac{50}{150} = 0.33$$

	Yes	No	
A	50	30	80
B	100	20	120
	150	50	200

Visualizing a Policy



Agent

- Agent consists of
 - States
 - Policy
 - **Value function**
 - Model

Value function

A function that determines the expected value of the state, given a policy

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, \pi] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s, \pi]\end{aligned}$$

$\gamma \in [0,1]$ is a discount factor

- A factor to balance between immediate reward and long-term reward

High value tells us that the state is more desirable

- If the state is more desirable, the agent can select action that results in that state.

Value function

Since the return can be written as $G_t = R_{t+1} + \gamma G_{t+1}$

We can rewrite the value function as

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, A_t \sim \pi(s)] \\ &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)] \end{aligned}$$

$A_t \sim \pi(s)$ means that we choose action A from policy π

*** This is called a **Bellman equation** ***

Value function

Bellman Equation

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[G_t \mid S_t = s, A_t \sim \pi(s)] \\ &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(s)] \end{aligned}$$

If we want to find the optimal value, we can modify the equation to

$$v_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

- Since this is an optimal solution, it does not depend on a policy.

Value function approximations

In many cases, we cannot find, or it is not feasible to find, the exact value from the value function.

- The states are infinitely many.

We need to approximate the function.

- More to come

Agent

- Agent consists of
 - States
 - Policy
 - Value function
 - Model

Model

A **model** predicts what will happen next in the environment

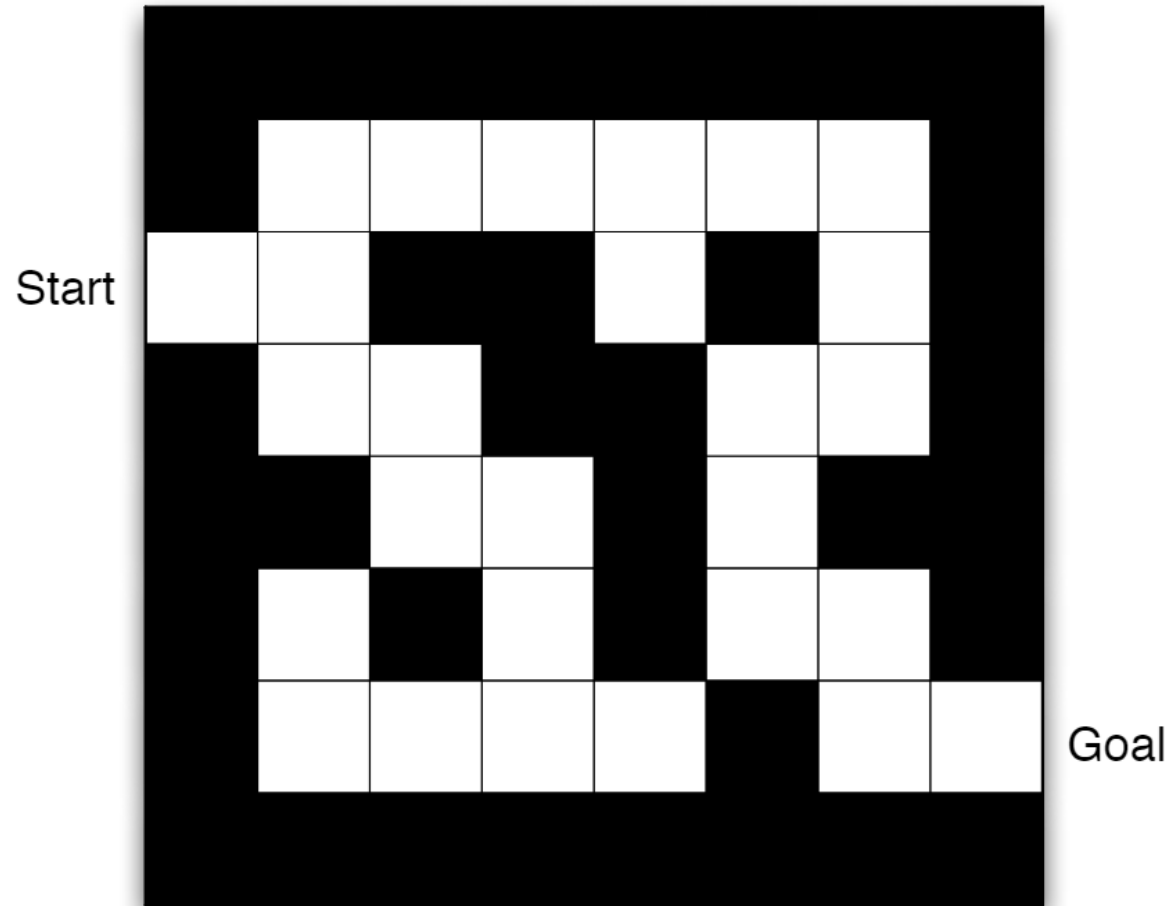
For example, a model P predicts the next state

$$P(s, a, s') \approx p(S_{t+1} = s' \mid S_t = s, A_t = a)$$

For example, a model R predicts the next reward

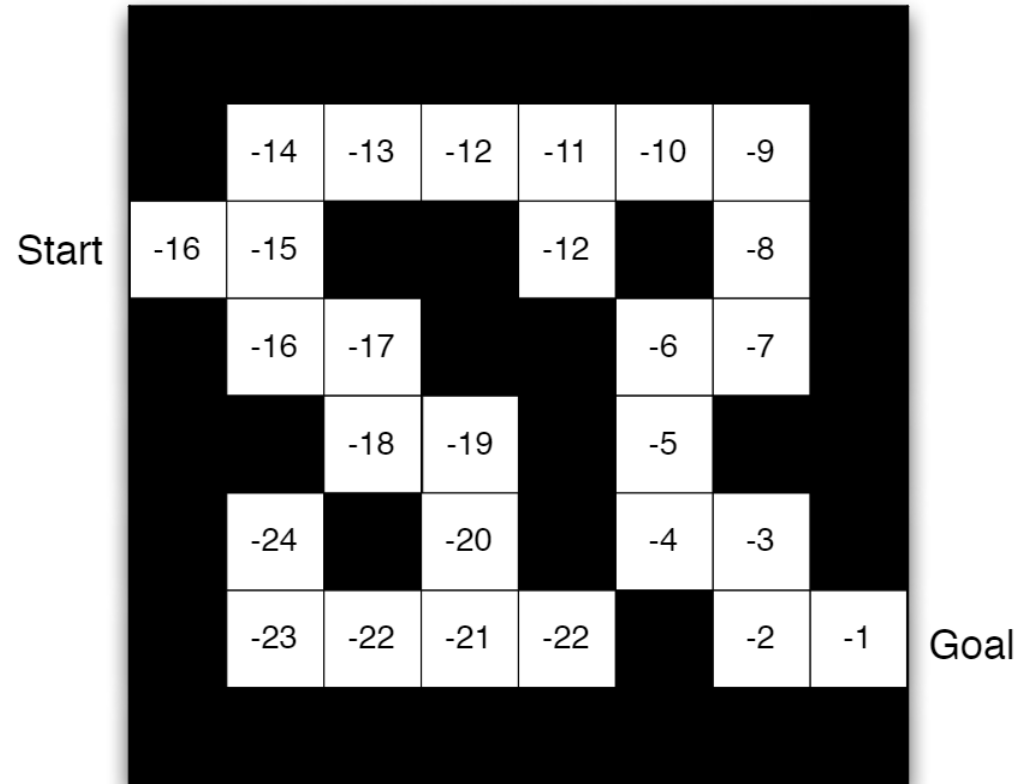
$$R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

Maze Example



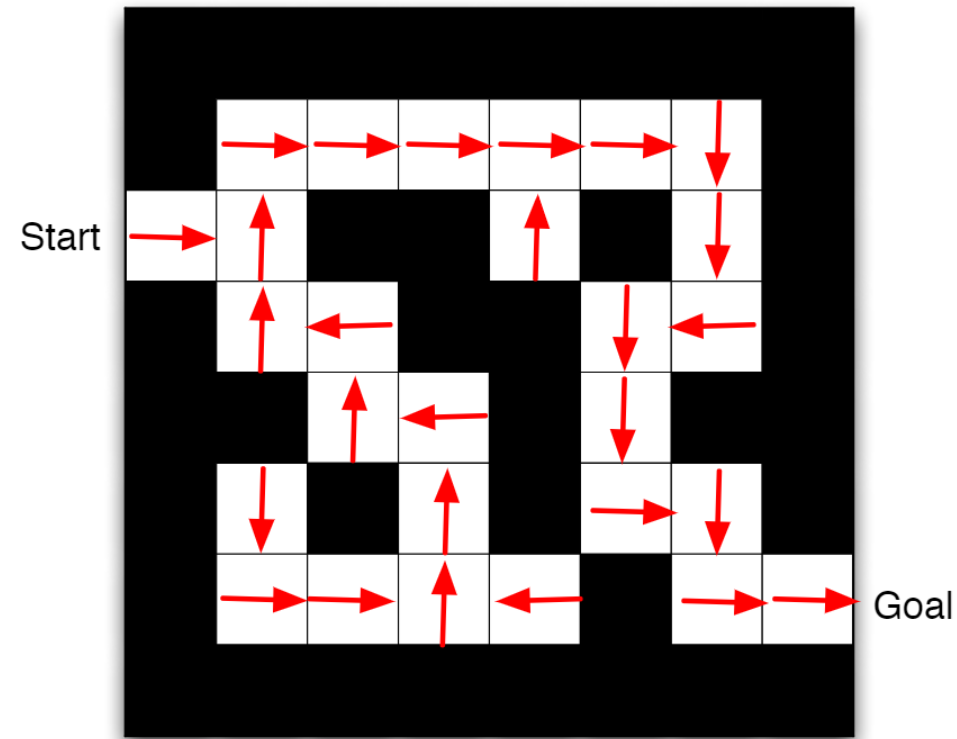
- ▶ Rewards: -1 per time-step
- ▶ Actions: N, E, S, W
- ▶ States: Agent's location

Maze Example: Value Function



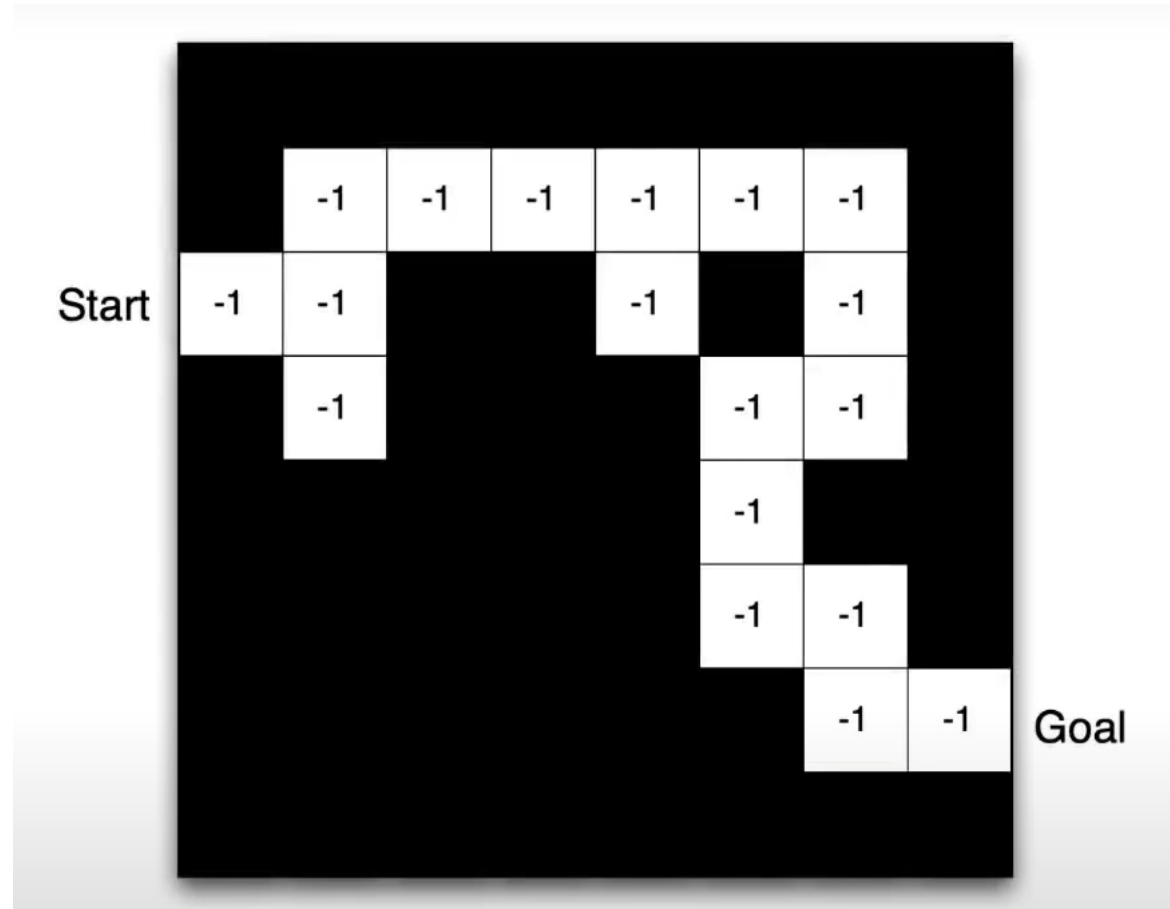
- Numbers represent value $v_{\pi}(s)$ of each state s

Maze Example: Policy



- Arrows represent policy $\pi(s)$ for each state s

Model



Agent Categories

- Value Based
 - No Policy
 - Value Function
- Policy Based
 - Policy
 - No Value Function
- Actor Critic
 - Policy
 - Value Function

Agent Categories

- Model Free
 - Policy and/or Value Function
 - No Model
- Model-based
 - (Optional) Policy and/or Value Function
 - Model

Lecture 1

Agent

Action

Environment

Reward

Next State



Agent

Action

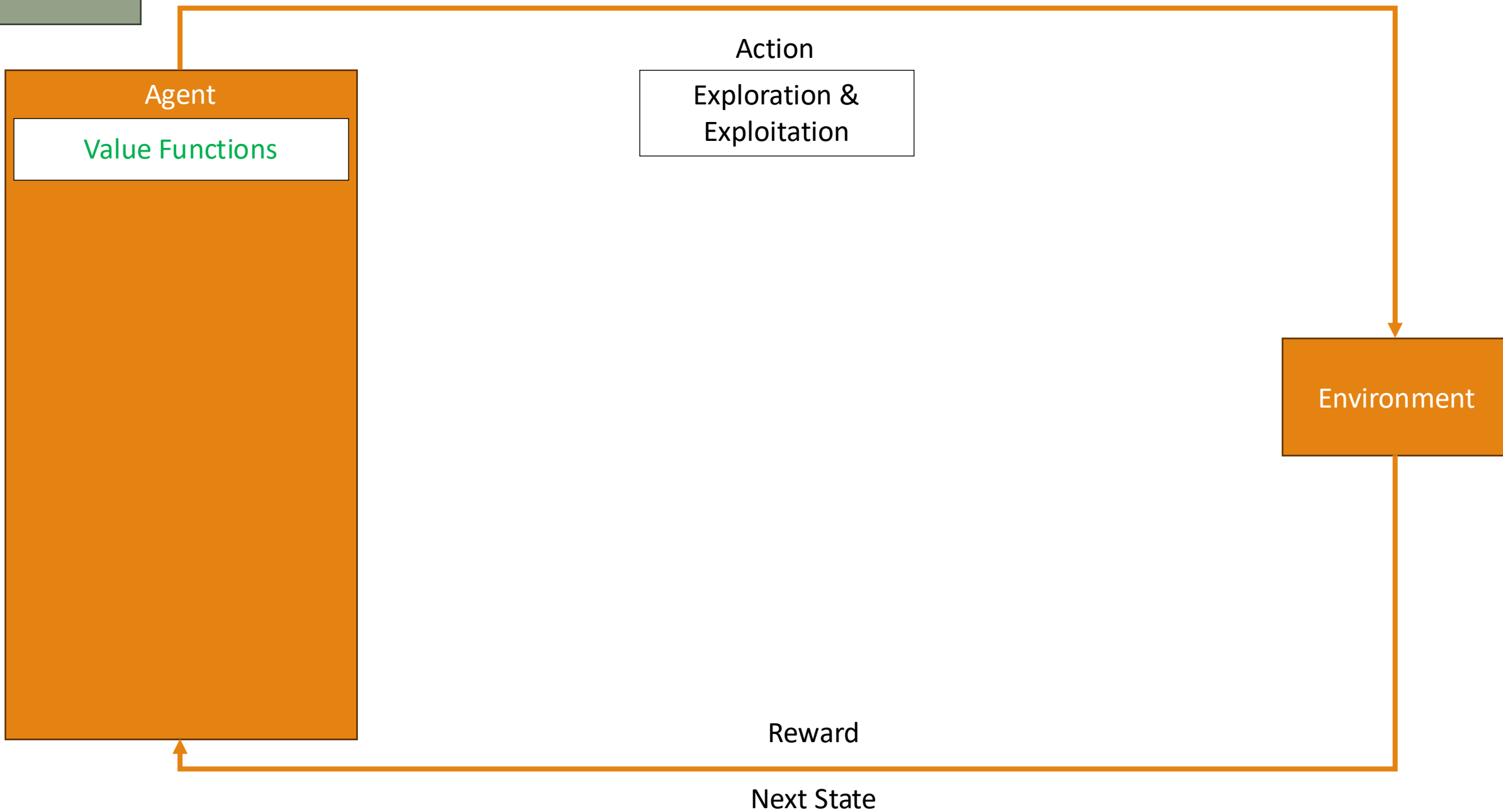
Exploration &
Exploitation

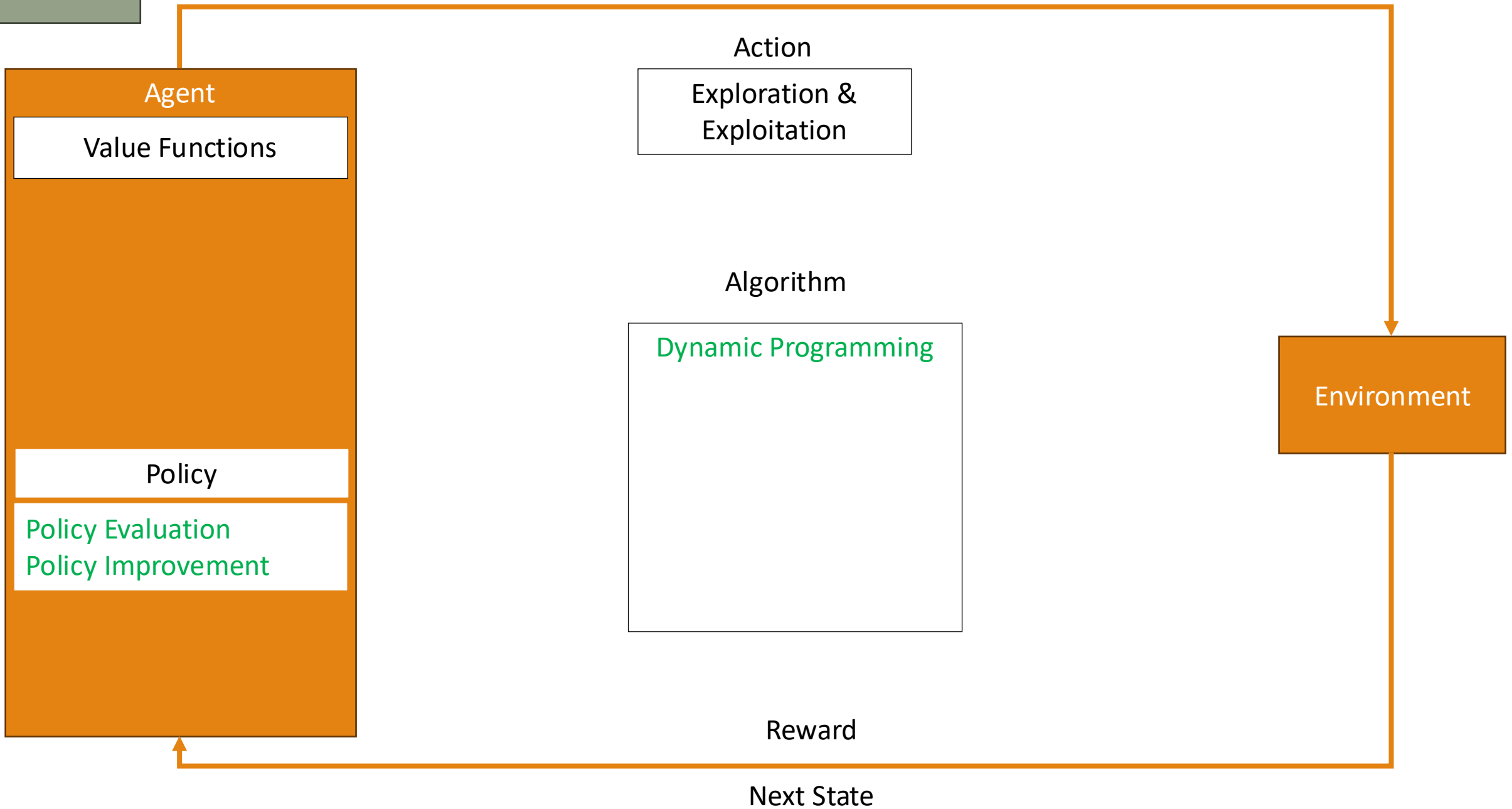
Environment

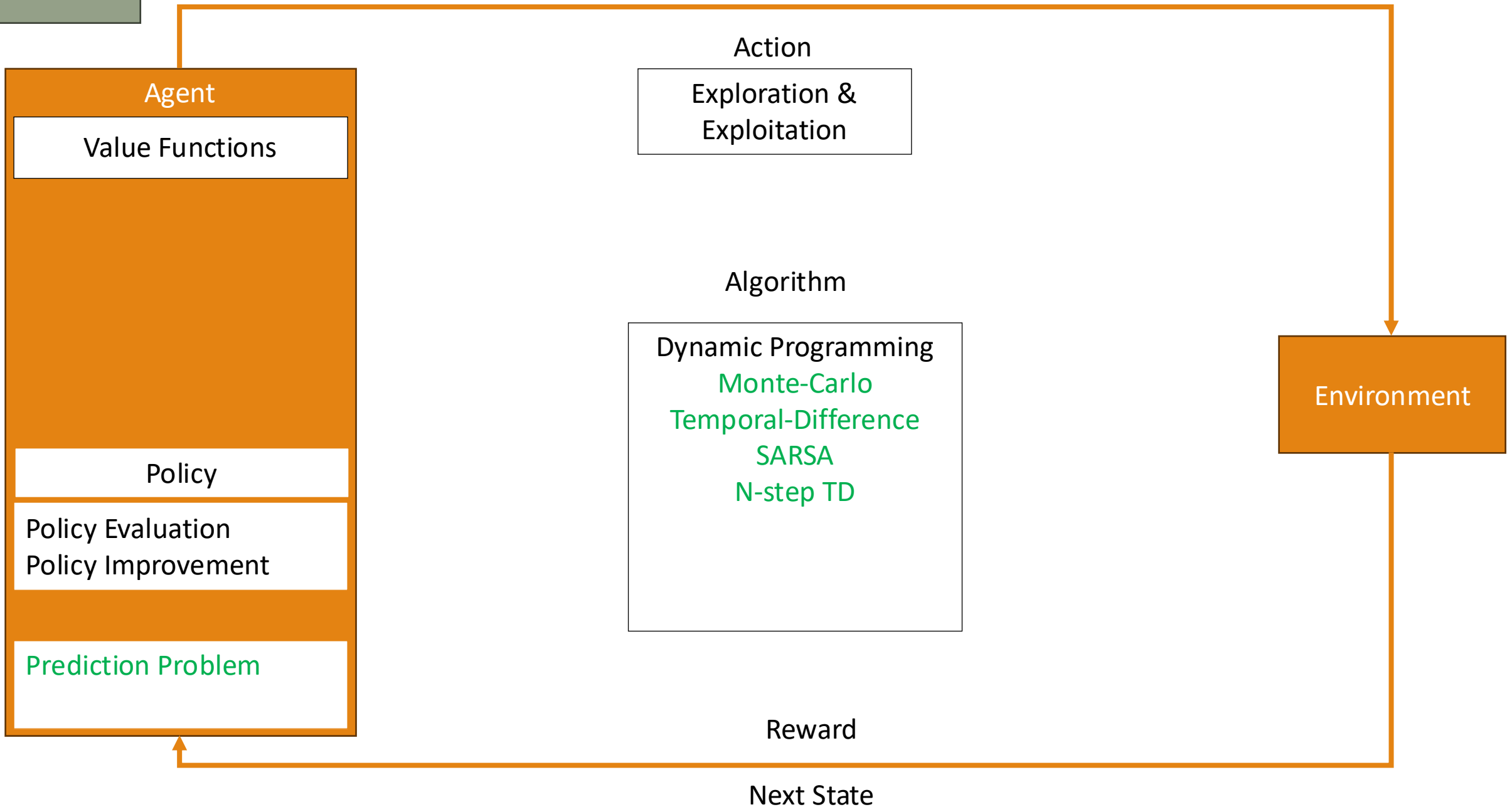
Reward

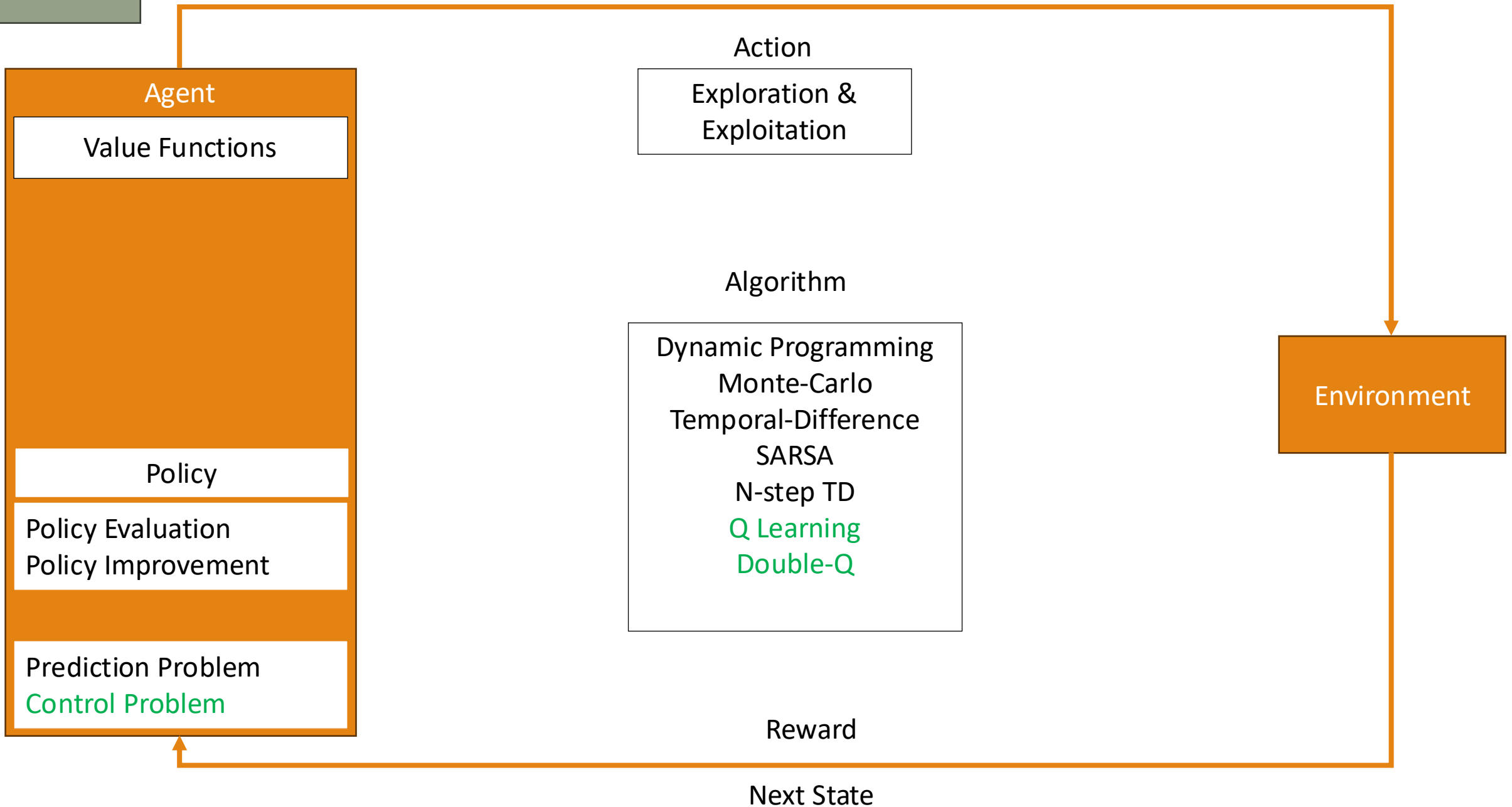
Next State

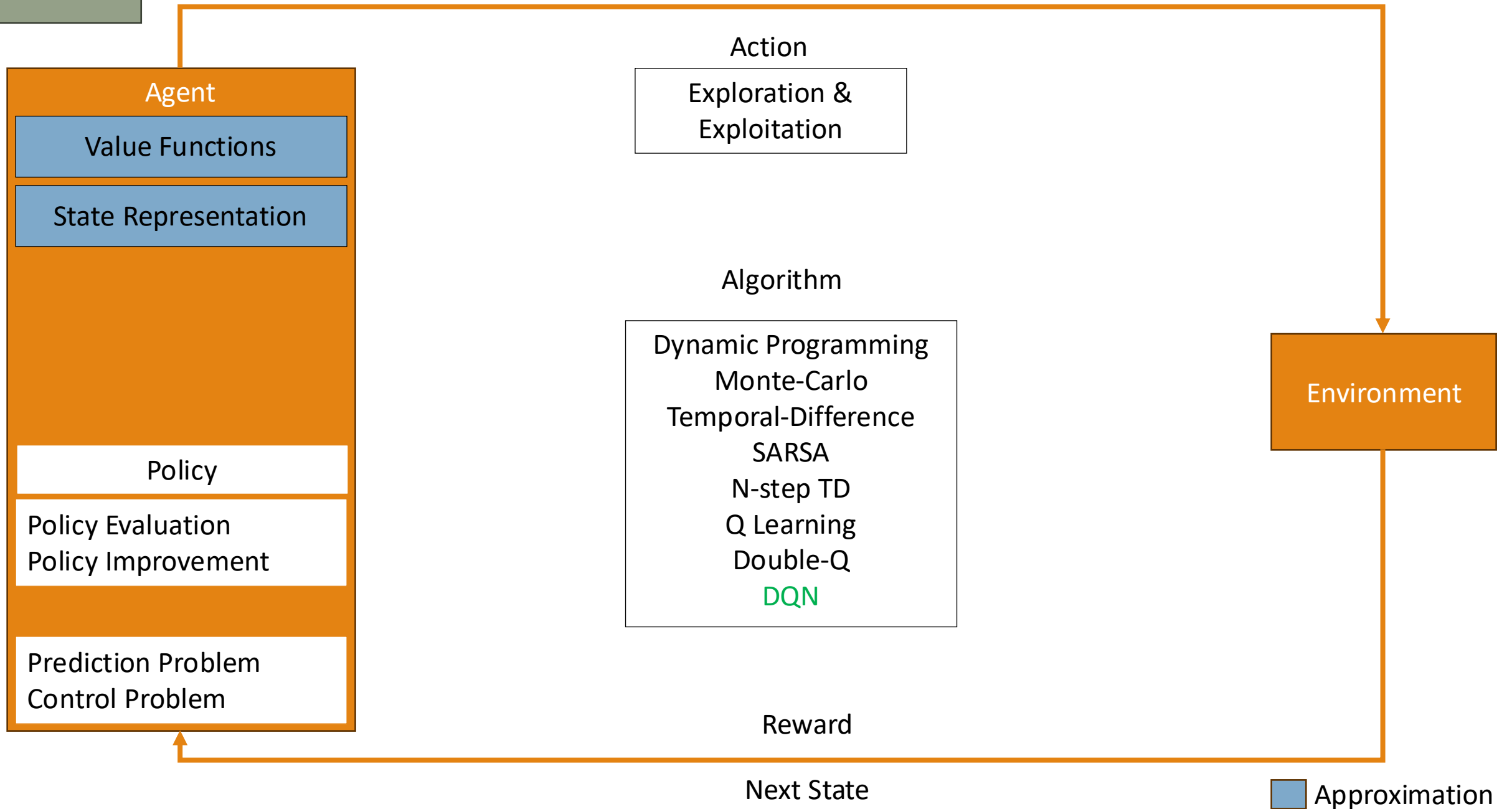


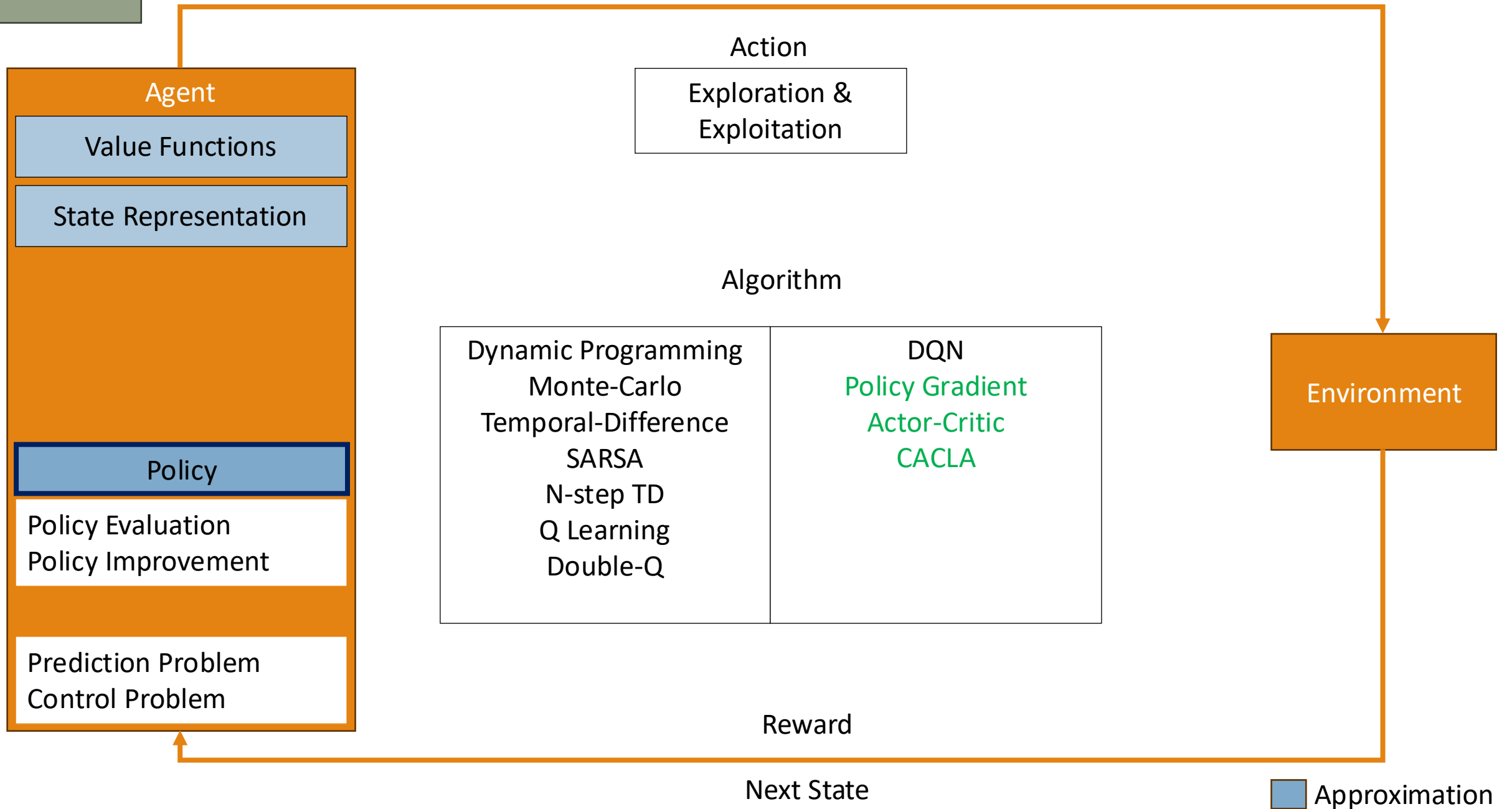












Agent

Value Functions

State Representation

Model

Policy

Policy Evaluation
Policy Improvement

Prediction Problem
Control Problem

Action

Exploration &
Exploitation

Algorithm

Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q

DQN
Policy Gradient
Actor-Critic
CACLA
Dyna-Q

Reward

Next State

Environment

 Approximation

